Information and Communication Technologies

Assignment 1

Design and Implementation of a Console-Based E-Commerce System Using Java

Project Report

Student: Kaisarbekov Argyn Zhasulanuly

Student ID: 4211

26 October 2026

1. Introduction and requirements

Objective

The objective of this project is to design and implement a simple console-based e-commerce system in Java. It provides practice in class design, encapsulation and inheritance through the management of users, products and orders.

Requirement analysis

This project is a small console shop written in Java. It has customers, employees, books, electronics, clothing and orders. Customers can buy products, and employees can add users and products and view all orders.

I kept the design close to the examples from the course so that the main ideas are easy to follow: private fields, constructors, inheritance, methods, loops and ArrayList.

Requirement	Implementation
Users	User is the parent class of Customer and Employee.
Products	Product is the parent class of Book, Electronics and Clothing.
Orders	Order stores its customer, a list of purchased products and the total.
Add and list records	Employee menu actions add users/products and show the lists.
Place and view orders	Customers place orders and view their history; employees view all orders.
Encapsulation	State fields are private, with getters and controlled setters.
Collections	Shop keeps users, products and orders in ArrayList.
Console input	Main uses Scanner and separate menus for the two roles.

Basic rules

A purchase needs a registered customer and at least one product. Each selection adds one unit. Selecting the same ID twice means buying two units of that product. The combined demand must fit the current stock.

Product prices and stock cannot be negative. The program checks every selected product before changing any stock. The data exists only during the current run.

2. Classes and responsibilities

Class	What it does
User	Stores ID, name, email, password and role.
Customer	Creates a User with the customer role.
Employee	Creates a User with the employee role.
Product	Stores ID, name, price and stock, and displays common details.
Book	Adds the author and overrides displayInfo().
Electronics	Adds the brand and overrides displayInfo().
Clothing	Adds the size and overrides displayInfo().
Order	Stores the customer, products, total and a ready-made receipt.
Shop	Holds the three lists and performs registration, login and checkout.
Main	Starts the application, reads input and displays the menus.

Inheritance and associations

A customer and an employee are both users, so their classes extend User. A book is a product, so Book extends Product. The other product types use the same relationship.

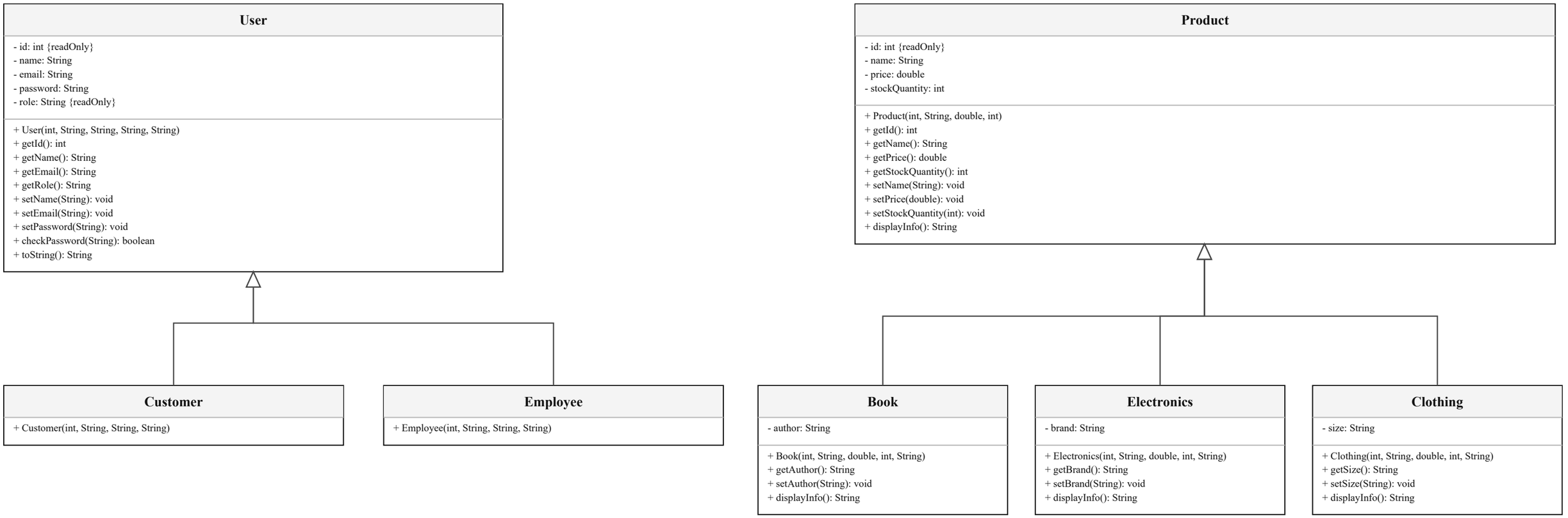
The subclass constructors call super(...) to initialise the common fields. Their displayInfo() methods call the parent implementation and add one detail, such as the book author.

An order is linked to its customer and catalogue products. It does not inherit from them. The UML diagrams on the next two pages show these links and the members of the application classes.

User and Product are ordinary parent classes in this version. An abstract class or an interface is not needed to demonstrate the inheritance required by the assignment.

Figure 1. Users and products

Open triangles point from a subclass to its parent.



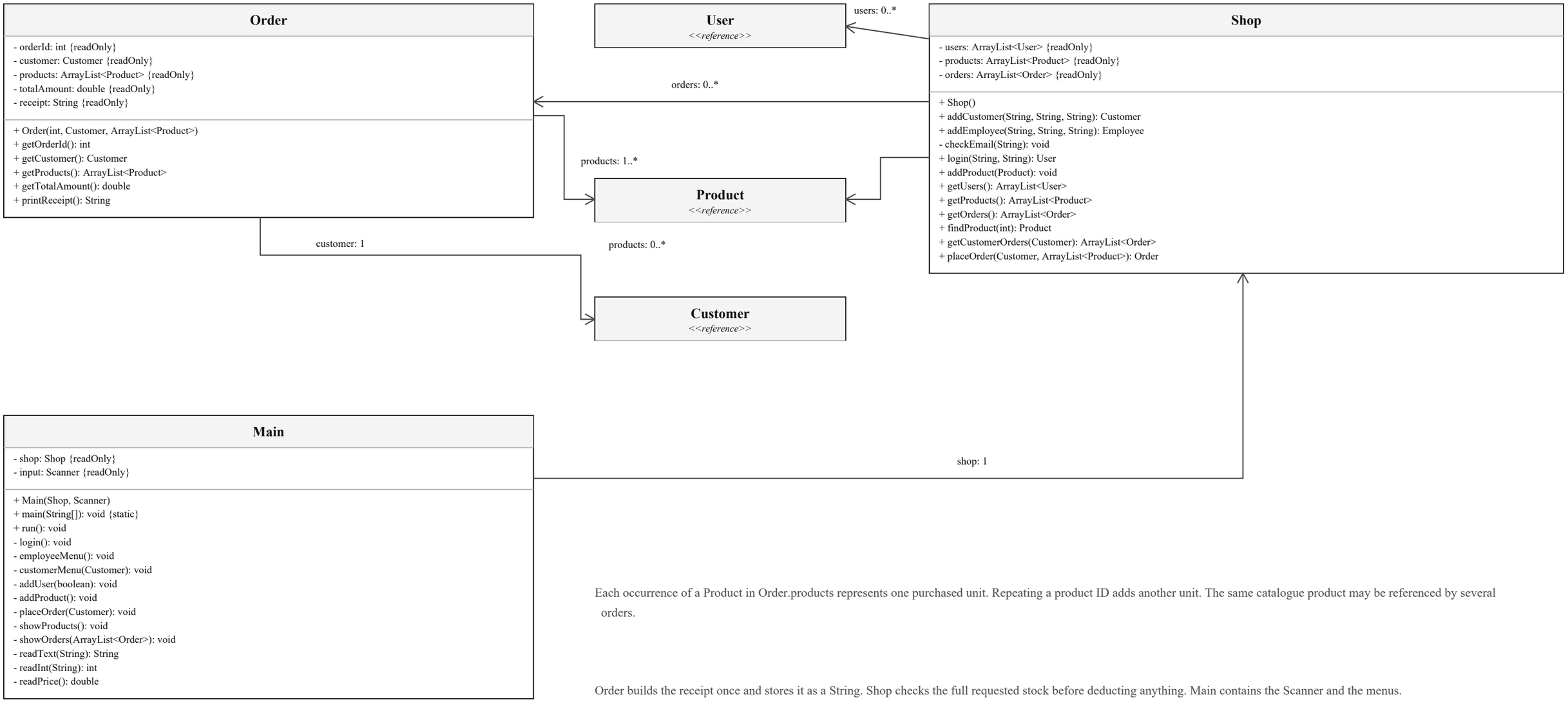
User and Product are ordinary parent classes. Customer and Employee pass their role to the User constructor. Each product subclass adds one field and overrides displayInfo().

Notation: + public, - private. {readOnly} means a final field, and {static} means a class member. Constructor parameters are shown by type. All declared state fields are private.

The model package contains the user, product and order classes. Shop coordinates operations, and Main handles console input.

Figure 2. Orders, shop and main program

Solid arrows show references stored in fields.



Each occurrence of a Product in Order.products represents one purchased unit. Repeating a product ID adds another unit. The same catalogue product may be referenced by several orders.

Order builds the receipt once and stores it as a String. Shop checks the full requested stock before deducting anything. Main contains the Scanner and the menus.

Scanner, ArrayList, String and other Java library types appear as field and parameter types. The optional test runner is separate from the application.

3. How the code works

Private fields and controlled changes

The fields are private. A getter reads a value, and a setter checks a new value before changing it. For example, setStockQuantity rejects a negative stock value. The ID and role are assigned when an object is created and cannot be reassigned through setters.

The base setters are final, so a subclass cannot replace their checks. Shop is also final because this program does not need a subclass of the shop.

Adding products and placing an order

Main reads the selected product ID and asks Shop to find the corresponding Product object. It adds that reference to an ArrayList. There is no separate cart class: one entry in this list means one unit.

For example, choosing IDs 1, 1 and 2 creates a list containing a book, the same book again, and a mouse. The customer enters 0 to finish and y to confirm.

Shop.placeOrder first checks the customer and the selected products. An outer loop visits each catalogue product; an inner loop counts how many times it appears in the selection. This count is compared with the stock. Only after every check succeeds does another loop deduct one unit for each selected entry.

Receipt and total

Order adds the price of every list entry to totalAmount. At the same time, a StringBuilder builds the receipt. The finished string is stored in receipt, so later catalogue changes do not rewrite the printed purchase details.

Prices use double, a type covered in the course. The receipt displays two decimal places. This is a small teaching example; exact financial calculations would be a reason to use BigDecimal in a later version.

Menus and login

Main contains the main, employee and customer menus. while repeats a menu, and switch chooses an action. Scanner reads whole lines. The numeric helpers repeat a question after incorrect input.

The classroom login compares the entered password with a String stored in memory. Passwords are not shown in user listings. This simple account handling is intended for the demonstration, not real user credentials.

4. Testing

The application was compiled and run on Windows using OpenJDK 27, with Java 17 selected as the compilation target. It has no external library dependencies.

All 104 checks in eight test groups passed. The main and test sources compiled with -Xlint:all without warnings.

Group	What was checked
Model and inheritance	Three product types, overridden output and invalid field values.
Accounts	Roles, login, duplicate email and invalid registration data.
Successful orders	Two books and one mouse: total 22000.00 KZT and correct stock.
Rejected orders	Repeated-product demand, empty selections and no partial stock changes.
Receipt and lists	Saved receipt/total and copies of the internal lists.
Console workflow	Adding all three product types and both user types, then purchasing.
Input and errors	Wrong numbers, unknown options, cancellation, stock errors and EOF.
Encapsulation	Reflection checks that state fields in the application classes are private.

The test runner is under src/test/java. It is separate from the application and is not included in the application JAR.

The packaged application was also launched through run.ps1. Selecting IDs 1, 1, 2 and confirming the purchase produced a 22000.00 KZT receipt. The remaining stock was 8 books and 4 mice.

The application works in one console thread and uses in-memory storage. The checks do not claim concurrent server behaviour. The POSIX scripts are included for portability but were not run on this Windows computer.

5. Running and demonstrating the program

Extract the new archive into an empty folder. On Windows, open run.cmd. To inspect the Java code, open the project folder in IntelliJ IDEA and run src/main/java/edu/ict/shop/Main.java.

The project includes an IntelliJ module, source roots and a run configuration named ICT Console Shop. This computer has the SDK openjdk-27. On another computer, select an installed JDK 17 or newer in Project Structure.

Role	Email	Password
Employee	employee@shop.local	Employee123
Customer	customer@shop.local	Customer123

ID	Product	Price (KZT)	Stock
1	Java Basics	3500.00	10
2	Wireless Mouse	15000.00	5
3	Campus T-Shirt	8000.00	8

Suggested demonstration

Log in as the employee, show the users and add a product. Log out, then log in as the customer. Choose Place order and enter 1, 1, 2, 0, pressing Enter after each number. Confirm with y.

Show the receipt, the customer's history and the remaining stock. Return to the employee menu and choose option 6 to view every order. An excessive repeated selection can be used to show rejection without stock changes.

Storage

Closing the program clears its in-memory records. The next run restores the sample users and products. The supplied scripts can rebuild the JAR and run the tests.

6. Conclusion

The application meets the assignment requirements. It uses the required user and product hierarchies, private fields, getters and setters, ArrayList and Scanner. The menus cover adding and listing records, placing orders and viewing them.

The code follows a short path: Main reads the input, Shop checks and saves the operation, and Order prepares the receipt. Using a plain list of selected products makes the order model easier to explain.

Possible extensions are saving data to a file or database, searching the catalogue and providing more editing actions in the menus. The current menus provide adding and listing; product setters demonstrate how a controlled edit would work.

References

1. Course assignment:
Assignment 1: Console-Based E-Commerce System using Java

2. Course Modules 2-4: Java basics, classes, collections, UML and inheritance.
Information and Communication Technologies repository

Appendix A. Complete application source

The following pages contain the complete application source. The original Java files, test runner, launch scripts, compiled JAR and editable UML are included in the project archive. The links above are the course materials; this project's implementation is supplied with the report.

Line numbers help identify code during the defence. Long lines may wrap on the page. The separate test runner is in src/test/java.

Appendix A - edu/ict/shop/Main.java

```
 1  package edu.ict.shop;
 2
 3  import java.util.ArrayList;
 4  import java.util.NoSuchElementException;
 5  import java.util.Scanner;
 6  import edu.ict.shop.model.*;
 7
 8  public class Main {
 9      private final Shop shop;
10      private final Scanner input;
11
12      public Main(Shop shop, Scanner input) {
13          this.shop = shop;
14          this.input = input;
15      }
16
17      public static void main(String[] args) {
18          new Main(new Shop(), new Scanner(System.in)).run();
19      }
20
21      public void run() {
22          System.out.println("=== ICT Console Shop ===");
23          System.out.println("Employee: employee@shop.local / Employee123");
24          System.out.println("Customer: customer@shop.local / Customer123");
25          try {
26              while (true) {
27                  System.out.println("\nMAIN MENU\n1. Log in\n2. Register customer\n3. List products\n0.
     Exit");
28                  int choice = readInt("Choose: ");
29                  if (choice == 0) { break; }
30                  try {
31                      switch (choice) {
32                          case 1: login(); break;
33                          case 2: addUser(false); break;
34                          case 3: showProducts(); break;
35                          default: System.out.println("Unknown option.");
36                      }
37                  } catch (IllegalArgumentException error) {
38                      System.out.println("Error: " + error.getMessage());
39                  }
40              }
41              System.out.println("Goodbye.");
42          } catch (NoSuchElementException error) {
43              System.out.println("\nInput closed. Goodbye.");
44          }
45      }
46
47      private void login() {
48          String email = readText("Email: ");
49          String password = readText("Password: ");
50          User user = shop.login(email, password);
51          if (user == null) {
52              System.out.println("Invalid email or password.");
53          } else if (user instanceof Employee) {
54              employeeMenu();
55          } else {
56              customerMenu((Customer) user);
57          }
58      }
59
60      private void employeeMenu() {
61          while (true) {
62              System.out.println("\nEMPLOYEE MENU\n1. List products\n2. Add product"
63                      + "\n3. List users\n4. Add customer\n5. Add employee\n6. View all orders\n0. Log
    out");
64              int choice = readInt("Choose: ");
65              if (choice == 0) { return; }
66              try {
67                  switch (choice) {
68                      case 1: showProducts(); break;
69                      case 2: addProduct(); break;
70                      case 3:
```

```
71                          for (User user : shop.getUsers()) { System.out.println(user); }
72                          break;
73                      case 4: addUser(false); break;
74                      case 5: addUser(true); break;
75                      case 6: showOrders(shop.getOrders()); break;
76                      default: System.out.println("Unknown option.");
77                  }
78              } catch (IllegalArgumentException error) {
79                  System.out.println("Error: " + error.getMessage());
80              }
81          }
82      }
83
84      private void customerMenu(Customer customer) {
85          while (true) {
86              System.out.println("\nCUSTOMER MENU\n1. List products\n2. Place order\n3. My orders\n0.
    Log out");
87              int choice = readInt("Choose: ");
88              if (choice == 0) { return; }
89              try {
90                  switch (choice) {
91                      case 1: showProducts(); break;
92                      case 2: placeOrder(customer); break;
93                      case 3: showOrders(shop.getCustomerOrders(customer)); break;
94                      default: System.out.println("Unknown option.");
95                  }
96              } catch (IllegalArgumentException error) {
97                  System.out.println("Error: " + error.getMessage());
98              }
99          }
100     }
101
102     private void addUser(boolean employee) {
103         String name = readText("Name: ");
104         String email = readText("Email: ");
105         String password = readText("Password (at least 4 characters): ");
106         User user;
107         if (employee) {
108             user = shop.addEmployee(name, email, password);
109         } else {
110             user = shop.addCustomer(name, email, password);
111         }
112         System.out.println("User added: " + user);
113     }
114
115     private void addProduct() {
116         int type = readInt("Type: 1 Book, 2 Electronics, 3 Clothing: ");
117         if (type < 1 || type > 3) {
118             throw new IllegalArgumentException("Choose a type from 1 to 3.");
119         }
120         int id = shop.getProducts().size() + 1;
121         String name = readText("Name: ");
122         double price = readPrice();
123         int stock = readInt("Stock: ");
124         Product product;
125         if (type == 1) {
126             product = new Book(id, name, price, stock, readText("Author: "));
127         } else if (type == 2) {
128             product = new Electronics(id, name, price, stock, readText("Brand: "));
129         } else {
130             product = new Clothing(id, name, price, stock, readText("Size: "));
131         }
132         shop.addProduct(product);
133         System.out.println("Product added: " + product.displayInfo());
134     }
135
136     private void placeOrder(Customer customer) {
137         ArrayList<Product> selected = new ArrayList<>();
138         showProducts();
139         System.out.println("One selection = one unit. Repeat an ID for more units. Enter 0 to
    finish.");
140         while (true) {
141             int id = readInt("Product ID: ");
142             if (id == 0) { break; }
143             Product product = shop.findProduct(id);
```

```
144             selected.add(product);
145             System.out.println("Added: " + product.getName());
146         }
147         if (selected.isEmpty()) {
148             System.out.println("Order cancelled: no products selected.");
149         } else if (readText("Confirm order? (y/n): ").equalsIgnoreCase("y")) {
150             Order order = shop.placeOrder(customer, selected);
151             System.out.println("Order placed.");
152             System.out.print(order.printReceipt());
153         } else {
154             System.out.println("Order cancelled.");
155         }
156     }
157
158     private void showProducts() {
159         if (shop.getProducts().isEmpty()) { System.out.println("No products."); }
160         for (Product product : shop.getProducts()) {
161             System.out.println(product.displayInfo());
162         }
163     }
164
165     private void showOrders(ArrayList<Order> orders) {
166         if (orders.isEmpty()) { System.out.println("No orders yet."); }
167         for (Order order : orders) { System.out.print(order.printReceipt()); }
168     }
169
170     private String readText(String prompt) {
171         while (true) {
172             System.out.print(prompt);
173             String value = input.nextLine().trim();
174             if (!value.isEmpty()) { return value; }
175             System.out.println("Enter a value.");
176         }
177     }
178
179     private int readInt(String prompt) {
180         while (true) {
181             try {
182                 return Integer.parseInt(readText(prompt));
183             } catch (NumberFormatException error) {
184                 System.out.println("Enter an integer.");
185             }
186         }
187     }
188
189     private double readPrice() {
190         while (true) {
191             try {
192                 double value = Double.parseDouble(readText("Price in KZT: "));
193                 if (Double.isFinite(value) && value >= 0) { return value; }
194             } catch (NumberFormatException ignored) {
195                 // Ask again after text that is not a number.
196             }
197             System.out.println("Enter a finite, non-negative price.");
198         }
199     }
200 }
```

Appendix A - edu/ict/shop/model/Book.java

```
1  package edu.ict.shop.model;
2
3  public class Book extends Product {
4      private String author;
5
6      public Book(int id, String name, double price, int stockQuantity, String author) {
7          super(id, name, price, stockQuantity);
8          setAuthor(author);
9      }
10
11     public String getAuthor() { return author; }
12
13     public final void setAuthor(String author) {
14         if (author == null || author.isBlank()) {
15             throw new IllegalArgumentException("Author must not be empty.");
16         }
17         this.author = author.trim();
18     }
19
20     @Override
21     public String displayInfo() {
22         return "Book | " + super.displayInfo() + " | author=" + author;
23     }
24 }
```

Appendix A - edu/ict/shop/model/Clothing.java

```
1  package edu.ict.shop.model;
2
3  public class Clothing extends Product {
4      private String size;
5
6      public Clothing(int id, String name, double price, int stockQuantity, String size) {
7          super(id, name, price, stockQuantity);
8          setSize(size);
9      }
10
11     public String getSize() { return size; }
12
13     public final void setSize(String size) {
14         if (size == null || size.isBlank()) {
15             throw new IllegalArgumentException("Size must not be empty.");
16         }
17         this.size = size.trim();
18     }
19
20     @Override
21     public String displayInfo() {
22         return "Clothing | " + super.displayInfo() + " | size=" + size;
23     }
24 }
```

Appendix A - edu/ict/shop/model/Customer.java

```
1  package edu.ict.shop.model;
2
3  public class Customer extends User {
4      public Customer(int id, String name, String email, String password) {
5          super(id, name, email, password, "Customer");
6      }
7  }
```

Appendix A - edu/ict/shop/model/Electronics.java

```
1  package edu.ict.shop.model;
2
3  public class Electronics extends Product {
4      private String brand;
5
6      public Electronics(int id, String name, double price, int stockQuantity, String brand) {
7          super(id, name, price, stockQuantity);
8          setBrand(brand);
9      }
10
11     public String getBrand() { return brand; }
12
13     public final void setBrand(String brand) {
14         if (brand == null || brand.isBlank()) {
15             throw new IllegalArgumentException("Brand must not be empty.");
16         }
17         this.brand = brand.trim();
18     }
19
20     @Override
21     public String displayInfo() {
22         return "Electronics | " + super.displayInfo() + " | brand=" + brand;
23     }
24 }
```

Appendix A - edu/ict/shop/model/Employee.java

```
1  package edu.ict.shop.model;
2
3  public class Employee extends User {
4      public Employee(int id, String name, String email, String password) {
5          super(id, name, email, password, "Employee");
6      }
7  }
```

Appendix A - edu/ict/shop/model/Order.java

```
 1  package edu.ict.shop.model;
 2
 3  import java.util.ArrayList;
 4  import java.util.Locale;
 5
 6  public class Order {
 7      private final int orderId;
 8      private final Customer customer;
 9      private final ArrayList<Product> products;
10      private final double totalAmount;
11      private final String receipt;
12
13      public Order(int orderId, Customer customer, ArrayList<Product> products) {
14          if (orderId <= 0 || customer == null || products == null || products.isEmpty()) {
15              throw new IllegalArgumentException("An order needs an ID, customer and products.");
16          }
17          this.orderId = orderId;
18          this.customer = customer;
19          this.products = new ArrayList<>(products);
20          double total = 0;
21          StringBuilder text = new StringBuilder();
22          text.append("\n=== RECEIPT #").append(orderId).append(" ===\n");
23          text.append("Customer: ").append(customer.getName()).append('\n');
24          for (Product product : this.products) {
25              if (product == null) {
26                  throw new IllegalArgumentException("An order must not contain a null product.");
27              }
28              total += product.getPrice();
29              text.append(String.format(Locale.ROOT, "%s - %.2f KZT%n",
30                      product.getName(), product.getPrice()));
31          }
32          if (!Double.isFinite(total)) {
33              throw new IllegalArgumentException("Order total is too large.");
34          }
35          this.totalAmount = total;
36          text.append(String.format(Locale.ROOT, "TOTAL: %.2f KZT%n", totalAmount));
37          this.receipt = text.toString();
38      }
39
40      public int getOrderId() { return orderId; }
41      public Customer getCustomer() { return customer; }
42      public ArrayList<Product> getProducts() { return new ArrayList<>(products); }
43      public double getTotalAmount() { return totalAmount; }
44      public String printReceipt() { return receipt; }
45  }
```

Appendix A - edu/ict/shop/model/Product.java

```
 1  package edu.ict.shop.model;
 2
 3  import java.util.Locale;
 4
 5  public class Product {
 6      private final int id;
 7      private String name;
 8      private double price;
 9      private int stockQuantity;
10
11      public Product(int id, String name, double price, int stockQuantity) {
12          if (id <= 0) {
13              throw new IllegalArgumentException("Product ID must be positive.");
14          }
15          this.id = id;
16          setName(name);
17          setPrice(price);
18          setStockQuantity(stockQuantity);
19      }
20
21      public int getId() { return id; }
22      public String getName() { return name; }
23      public double getPrice() { return price; }
24      public int getStockQuantity() { return stockQuantity; }
25
26      public final void setName(String name) {
27          if (name == null || name.isBlank()) {
28              throw new IllegalArgumentException("Product name must not be empty.");
29          }
30          this.name = name.trim();
31      }
32
33      public final void setPrice(double price) {
34          if (!Double.isFinite(price) || price < 0) {
35              throw new IllegalArgumentException("Price must be a finite, non-negative number.");
36          }
37          this.price = price;
38      }
39
40      public final void setStockQuantity(int stockQuantity) {
41          if (stockQuantity < 0) {
42              throw new IllegalArgumentException("Stock must not be negative.");
43          }
44          this.stockQuantity = stockQuantity;
45      }
46
47      public String displayInfo() {
48          return String.format(Locale.ROOT, "%d | %s | %.2f KZT | stock=%d",
49                  id, name, price, stockQuantity);
50      }
51  }
```

Appendix A - edu/ict/shop/model/User.java

```
 1  package edu.ict.shop.model;
 2
 3  public class User {
 4      private final int id;
 5      private String name;
 6      private String email;
 7      private String password;
 8      private final String role;
 9
10      public User(int id, String name, String email, String password, String role) {
11          if (id <= 0) {
12              throw new IllegalArgumentException("User ID must be positive.");
13          }
14          this.id = id;
15          setName(name);
16          setEmail(email);
17          setPassword(password);
18          this.role = role;
19      }
20
21      public int getId() { return id; }
22      public String getName() { return name; }
23      public String getEmail() { return email; }
24      public String getRole() { return role; }
25
26      public final void setName(String name) {
27          if (name == null || name.isBlank()) {
28              throw new IllegalArgumentException("Name must not be empty.");
29          }
30          this.name = name.trim();
31      }
32
33      public final void setEmail(String email) {
34          if (email == null || !email.contains("@")) {
35              throw new IllegalArgumentException("Email must contain @.");
36          }
37          this.email = email.trim();
38      }
39
40      public final void setPassword(String password) {
41          if (password == null || password.length() < 4) {
42              throw new IllegalArgumentException("Password must have at least 4 characters.");
43          }
44          this.password = password;
45      }
46
47      public boolean checkPassword(String enteredPassword) {
48          return password.equals(enteredPassword);
49      }
50
51      @Override
52      public String toString() {
53          return id + " | " + role + " | " + name + " | " + email;
54      }
55  }
```

Appendix A - edu/ict/shop/Shop.java

```
 1  package edu.ict.shop;
 2
 3  import java.util.ArrayList;
 4  import edu.ict.shop.model.*;
 5
 6  public final class Shop {
 7      private final ArrayList<User> users = new ArrayList<>();
 8      private final ArrayList<Product> products = new ArrayList<>();
 9      private final ArrayList<Order> orders = new ArrayList<>();
10
11      public Shop() {
12          addEmployee("Demo Employee", "employee@shop.local", "Employee123");
13          addCustomer("Demo Customer", "customer@shop.local", "Customer123");
14          addProduct(new Book(1, "Java Basics", 3500, 10, "ICT Teaching Team"));
15          addProduct(new Electronics(2, "Wireless Mouse", 15000, 5, "StudyTech"));
16          addProduct(new Clothing(3, "Campus T-Shirt", 8000, 8, "M"));
17      }
18
19      public Customer addCustomer(String name, String email, String password) {
20          checkEmail(email);
21          Customer customer = new Customer(users.size() + 1, name, email, password);
22          users.add(customer);
23          return customer;
24      }
25
26      public Employee addEmployee(String name, String email, String password) {
27          checkEmail(email);
28          Employee employee = new Employee(users.size() + 1, name, email, password);
29          users.add(employee);
30          return employee;
31      }
32
33      private void checkEmail(String email) {
34          if (email == null) {
35              throw new IllegalArgumentException("Email is required.");
36          }
37          for (User user : users) {
38              if (user.getEmail().equalsIgnoreCase(email.trim())) {
39                  throw new IllegalArgumentException("This email is already registered.");
40              }
41          }
42      }
43
44      public User login(String email, String password) {
45          if (email == null) { return null; }
46          for (User user : users) {
47              if (user.getEmail().equalsIgnoreCase(email.trim()) && user.checkPassword(password)) {
48                  return user;
49              }
50          }
51          return null;
52      }
53
54      public void addProduct(Product product) {
55          if (product == null) {
56              throw new IllegalArgumentException("Product is required.");
57          }
58          for (Product existing : products) {
59              if (existing.getId() == product.getId()) {
60                  throw new IllegalArgumentException("Product ID already exists.");
61              }
62          }
63          products.add(product);
64      }
65
66      public ArrayList<User> getUsers() { return new ArrayList<>(users); }
67      public ArrayList<Product> getProducts() { return new ArrayList<>(products); }
68      public ArrayList<Order> getOrders() { return new ArrayList<>(orders); }
69
70      public Product findProduct(int id) {
71          for (Product product : products) {
72              if (product.getId() == id) { return product; }
```

```
        }
        throw new IllegalArgumentException("Product was not found.");
    }

    public ArrayList<Order> getCustomerOrders(Customer customer) {
        ArrayList<Order> result = new ArrayList<>();
        for (Order order : orders) {
            if (order.getCustomer() == customer) { result.add(order); }
        }
        return result;
    }

    public Order placeOrder(Customer customer, ArrayList<Product> selected) {
        if (customer == null || !users.contains(customer)) {
            throw new IllegalArgumentException("Choose a registered customer.");
        }
        if (selected == null || selected.isEmpty()) {
            throw new IllegalArgumentException("The order is empty.");
        }
        for (Product product : selected) {
            if (!products.contains(product)) {
                throw new IllegalArgumentException("The order contains an unknown product.");
            }
        }

        // Each occurrence in the list means one unit of that product.
        for (Product product : products) {
            int wanted = 0;
            for (Product chosen : selected) {
                if (chosen == product) { wanted++; }
            }
            if (wanted > product.getStockQuantity()) {
                throw new IllegalArgumentException("Not enough stock for " + product.getName());
            }
        }

        // Nothing is deducted until every product has passed the checks.
        Order order = new Order(orders.size() + 1, customer, selected);
        for (Product product : selected) {
            product.setStockQuantity(product.getStockQuantity() - 1);
        }
        orders.add(order);
        return order;
    }
}
```